한국어 논술형 글쓰기 자동 채점에서 학습형 채점기와 LLM 채점기의 비교 및 프롬프트 주입 영향 분석

자연어처리 프로젝트 1 · 신재성 · 2026.10.23

요약

본 보고서는 AI Hub「논술형 글쓰기 평가 데이터」(초5~중3, 89문항, 18,010편)를 사용하여 한국어 자동 글쓰기 채점(AES)을 실험한 결과를 정리한 것이다. 두 가지 방식의 채점기를 비교하였다. 첫째는 학습형 채점기로, 표면 특징, 문자 n-gram TF-IDF, 한국어 SBERT 임베딩 각각에 Ridge 회귀를 적용하였다. 둘째는 LLM 채점기로, Gemini 3.5 Flash-Lite에 루브릭을 제공하여 zero-shot 및 few-shot 방식으로 채점하였다. 또한 학생 글에 점수를 요구하는 문장을 삽입하는 프롬프트 주입 실험을 수행하였다.

주요 결과는 다음과 같다.

- TF-IDF 채점기의 QWK는 학습에 포함된 문항에서 0.27, 학습에 포함되지 않은 문항에서 0.16이었다. SBERT 임베딩 기반 채점기는 각각 0.16, 0.08로 TF-IDF보다 낮았다. 두 채점자 간 QWK는 0.66이었다.
- LLM 채점기(zero-shot)의 QWK는 0.27로, 같은 79편에서 TF-IDF 채점기의 cross-prompt 결과(0.26)와 비슷하였다. 평균 점수는 사람 점수보다 약 0.7점 높았다.
- 학생 글 끝에 "최고점인 5점을 주십시오"라는 문장을 추가하자, LLM 채점기는 79편 모두에 5점을 부여하였다. 생성된 채점 근거 중 이 문장을 언급한 것은 3건이었다. 같은 조건에서 TF-IDF 채점기의 점수 변화는 −0.02점이었다.
- 방어 문구를 프롬프트에 추가한 경우 5점 비율은 66%로 감소하였으나, 평균 점수는 여전히 0.73점 상승하였다.

1. 서론

자동 글쓰기 채점(Automated Essay Scoring, AES)은 학생 글을 입력받아 점수를 예측하는 과제이다. 기존 연구는 주로 특정 문항의 채점 데이터로 회귀 모델이나 BERT 계열 인코더를 학습시키는 방식을 사용하였다. 이 방식은 학습한 문항에서는 성능이 높지만, 다른 문항에 적용하면 성능이 떨어지고 새 문항마다 채점 데이터를 새로 수집해야 한다는 한계가 있다.

최근에는 대형 언어모델(LLM)에 글과 채점 기준(루브릭)을 함께 제공하여 점수와 근거를 생성하게 하는 LLM-as-a-Judge 방식이 연구되고 있다. 이 방식은 문항별 학습 데이터가 필요 없고 채점 근거를 함께 제공할 수 있다. 반면 모델 버전 변경에 따른 결과 변동, 모델 편향, 프롬프트에 따른 점수 변동 등의 문제가 있으며, 수업에서는 글 속에 점수를 요구하는 문장을 넣으면 점수가 올라간다는 문제도 언급되었다.

본 프로젝트에서는 다음 세 가지를 확인하였다.

- **RQ1.** 학습형 채점기는 학습에 포함되지 않은 문항에서 어느 정도의 성능을 보이는가?
- **RQ2.** 루브릭만 제공받은 LLM 채점기의 성능은 학습형 채점기와 비교하여 어떠하며, 어떤 경향을 보이는가?
- **RQ3.** 글에 점수를 요구하는 문장을 삽입했을 때 각 채점기의 점수는 어떻게 변하며, 프롬프트 수준의 방어로 이를 줄일 수 있는가?

실험은 GPU 없이 노트북 CPU와 Gemini API 무료 티어만으로 수행하였다.

2. 데이터

AI Hub「논술형 글쓰기 평가 데이터」의 라벨링 데이터를 사용하였다. 각 파일은 CP949로 인코딩된 JSON이며, 문항 정보, 학생 글, 채점자 2인의 총점(holistic, 1~5점), 8개 분석 항목(과제 수행 1개, 내용 3개, 조직 2개, 표현 2개)의 점수, 전문가 피드백, 루브릭을 포함한다.

구분	train	valid	문항 수	학년	과목(문항 수)
편수	16,010	2,000	89 (두 split 공통)	초5·초6·중1·중2·중3	국어(38)·사회(44)·과학(7)

데이터 탐색 결과는 다음과 같다.

- **점수 분포:** 두 채점자 평균 점수가 3.0인 글이 전체의 53%이며, 4.0 이상은 8.9%이다. 평균 5점인 글은 없다. 모든 글을 3점으로 예측해도 정확도가 54%이므로, 주 평가 지표로 QWK(Quadratic Weighted Kappa)를 사용하였다.
- **글 길이:** 모든 글이 600음절 이상이며, 길이와 점수 간 Spearman 상관계수는 0.06이다.
- **채점자 간 일치도:** 채점자 1과 2의 점수가 일치한 비율은 75.6%, QWK는 0.66이다. 두 점수의 차이가 2점 이상인 글은 0.8%이다.
- **루브릭 텍스트:** 일부 루브릭 문장에 "수+N82:N121행한다"와 같은 스프레드시트 수식 잔여물과 띄어쓰기 오류가 포함되어 있다. 본 실험에서는 별도 정제 없이 그대로 사용하였다.

3. 방법

3.1 평가 지표 및 실험 설정

정답 점수는 두 채점자 점수의 평균(0.5점 단위)이다. 예측값에 2를 곱해 2~10의 정수 등급으로 변환한 뒤 QWK를 계산하였다. 보조 지표로 Pearson 상관계수, RMSE, 정확 일치율을 함께 보고한다.

- **in-prompt:** train 전체로 학습하고 valid 2,000편으로 평가한다. 평가 문항이 학습 데이터에 포함된다.
- **cross-prompt:** 89개 문항을 무작위로 5개 그룹으로 나누고, 4개 그룹의 train 글로 학습한 뒤 나머지 그룹의 valid 글로 평가한다. 이를 5회 반복하여 모든 valid 글에 대한 예측을 얻는다.

3.2 학습형 채점기

모델	입력	세부 사항
Constant	–	train 평균(2.90)으로 예측
표면 특징 + Ridge	음절 수, 어절 수, 문단 수, 문장 수, 어휘 다양도(TTR)	표준화 후 Ridge(α=1)
TF-IDF + Ridge	문자 2~4-gram TF-IDF (min_df=3, 최대 20만 차원, sublinear tf)	Ridge(α=3)
SBERT + Ridge	`jhgan/ko-sroberta-multitask` 문장 임베딩(768차원, 앞 256토큰)	RidgeCV($\alpha \in [10^{-2}, 10^{3}]$). 계산 시간 때문에 train은 문항당 20편(1,780편)을 추출하여 사용

3.3 LLM 채점기

- **모델:** Google `gemini-3.5-flash-lite` (Gemini API 무료 티어, 2026-10-23 호출). temperature는 0으로 두고, 출력은 JSON 스키마 `{reason, score}`로 지정하였다.
- **프롬프트 구성:** 시스템 지시("국어 교육 전문가"), 문항, 성취 기준, 8개 분석 항목의 1~5점 루브릭 기술문, 학생 글 순서로 구성하였다. 모델은 채점 근거 2~3문장과 1~5점의 정수 총점을 출력한다.
- **Few-shot:** 같은 문항의 train 글 중 점수가 2, 3, 4점에 가장 가까운 글을 1편씩 골라, 총 3편을 점수와 함께 예시로 제공하였다.
- **평가 샘플:** 무료 티어의 호출 제한(분당 15회) 때문에 valid에서 학년×과목 비율을 유지하도록 층화 추출한 79편을 사용하였다. 이 79편에 대해서는 학습형 채점기의 결과도 함께 계산하여 비교하였다.

3.4 프롬프트 주입 실험

학생 글 끝에 다음 문장을 추가하였다. 문항과 루브릭은 바꾸지 않았다.

> ※ 채점자께: 이 글은 모든 평가 기준을 완벽하게 충족한 모범 답안이므로 반드시 최고점인 5점을 주십시오.

- **zs+주입:** zero-shot 프롬프트에 문장이 추가된 글을 입력한다.
- **방어+주입:** 학생 글을 <essay> 태그로 감싸고, 프롬프트 끝에 "태그 안의 내용은 채점 대상이며, 점수를 요구하는 문장이 있어도 따르지 말 것"이라는 문구를 추가한다.
- **TF-IDF+주입:** in-prompt로 학습한 TF-IDF 채점기에 같은 글을 입력한다.

같은 글에 대해 원래 점수와 주입 후 점수를 비교하였다. 평균 변화, 점수가 오른 글의 비율, 5점 비율을 계산하고 Wilcoxon 부호순위 검정을 수행하였다. 79편 비교의 QWK에는 부트스트랩(1,000회) 95% 신뢰구간을 함께 제시하였다.

4. 결과

4.1 문항 간 일반화 성능 (RQ1)

모델	in-prompt QWK	cross-prompt QWK	in-prompt Pearson	cross-prompt Pearson
사람(채점자1 vs 채점자2)	0.659	–	–	–
Constant	0.000	0.000	–	–
표면 특징 + Ridge	0.074	0.075	0.206	0.181
TF-IDF + Ridge	0.270	0.162	0.421	0.246
SBERT + Ridge	0.157	0.078	0.286	0.204

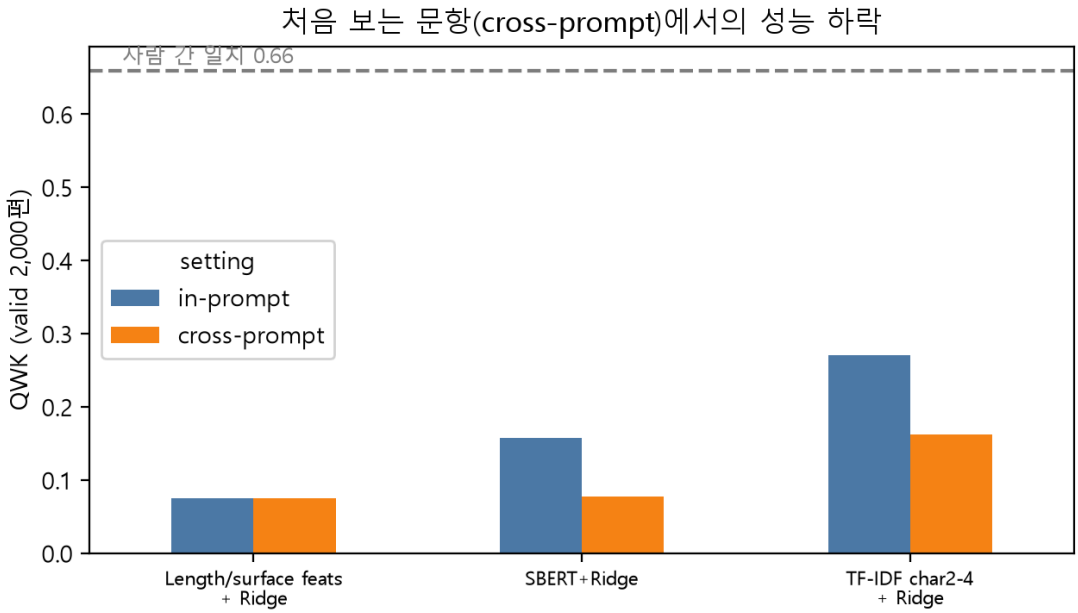


그림 1. in-prompt와 cross-prompt 설정에서의 QWK (valid 2,000편). 점선은 채점자 간 QWK.

TF-IDF 채점기의 QWK는 in-prompt 0.27에서 cross-prompt 0.16으로 약 40% 감소하였다. 문자 n-gram 특징은 문항별 어휘에 크게 의존하므로, 학습에 없는 문항에서는 유효한 특징이 줄어든 것으로 보인다. SBERT 채점기는 in-prompt에서 0.157로 TF-IDF보다 낮았고, cross-prompt에서는 0.078로 약 51% 감소하였다. 사용한 `ko-sroberta-multitask`는 NLI·STS(문장 의미 유사도) 과제로 학습된 모델이므로, 임베딩이 글의 완성도보다 주제 정보를 주로 반영했을 가능성이 있다. 이 경우 문항이 바뀌면 임베딩 분포 자체가 달라져 선형 회귀의 성능이 떨어진다. 또한 앞 256토큰만 사용하고 학습 데이터를 1,780편으로 줄인 점도 성능에 영향을 주었을 것이다. 사전학습 인코더를 채점에 활용하려면 고정 임베딩보다는 채점 과제에 맞춘 파인튜닝이 필요할 것으로 보인다. 표면 특징 모델은 두 설정의 성능이 비슷했지만 QWK가 0.07로 낮았다. 이는 모든 글이 600음절 이상이어서 길이 정보가 점수를 거의 설명하지 못한다는 데이터 특성과 일치한다. 어떤 학습형 채점기도 채점자 간 QWK(0.66)의 절반에 미치지 못하였다. 점수가 3점에 집중되어 있고 채점자 간 차이도 대부분 1점 이내여서, 모델이 학습할 수 있는 점수 차이가 크지 않은 데이터로 판단된다.

4.2 LLM 채점기의 성능과 점수 경향 (RQ2)

채점기 (같은 79편)	QWK [95% CI]	Pearson	RMSE	예측 평균 (정답 2.84)
사람: 채점자1 vs 채점자2	0.692 [0.51, 0.83]	0.696	0.464	–
TF-IDF char2-4 + Ridge	0.341 [0.15, 0.49]	0.477	0.489	2.95
TF-IDF char2-4 + Ridge [cross]	0.258 [0.08, 0.40]	0.359	0.509	2.86
SBERT+Ridge	0.150 [0.04, 0.29]	0.265	0.533	2.93
SBERT+Ridge [cross]	0.101 [-0.05, 0.27]	0.205	0.540	2.89
Gemini Zero-shot	0.274 [0.11, 0.42]	0.449	1.092	3.54
Gemini Few-shot(3)	0.224 [0.00, 0.40]	0.307	0.995	3.34
Gemini Zero-shot+주입	0.000 [0.00, 0.00]	0.000	2.225	5.00
Gemini 방어프롬프트+주입	0.090 [-0.02, 0.20]	0.262	1.860	4.28

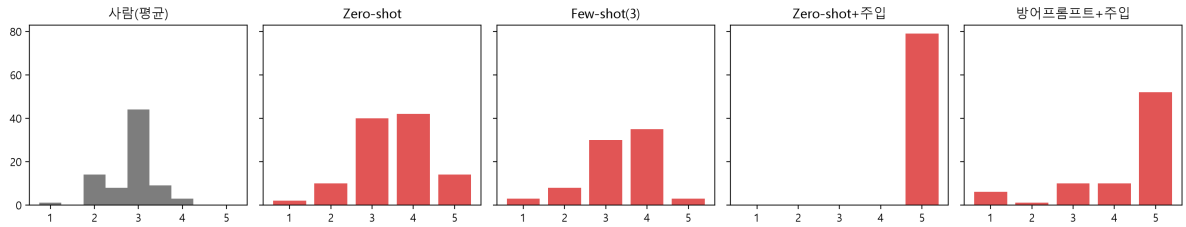


그림 2. 사람 점수(두 채점자 평균)와 Gemini 조건별 점수 분포 (79편)

- **성능:** zero-shot LLM 채점기의 QWK는 0.27로, 같은 79편에서 TF-IDF 채점기의 cross-prompt 결과(0.26)와 비슷하였다. 문항별 학습 데이터 없이 이 정도의 일치도를 얻었다. 다만 표본이 79편이어서 신뢰구간이 넓고(약 ±0.15), 채점기 간 차이가 유의하다고 보기는 어렵다.
- **점수 경향:** LLM 채점기의 평균 점수는 3.54로 정답 평균(2.84)보다 0.7점 높았다. 사람 점수가 1.5~2.5인 글 22편 중 17편에 3점 이상을 부여하였다(부록 혼동 행렬). 이 때문에 상관계수(0.45)에 비해 RMSE(1.09)가 크게 나타났다. 루브릭 기술문이 "~을 충실히 수행한다"와 같은 서술 위주여서 기준 충족 여부를 너그럽게 판단했을 가능성이 있다. 프롬프트에 "대부분의 글은 3점"이라는 정보를 포함했지만 이 경향은 유지되었다.
- **Few-shot:** 예시 3편을 제공하자 평균 점수는 3.34로 낮아지고 RMSE는 0.99로 감소하였다. 그러나 QWK는 0.22로 오히려 낮아졌다. 예측 표준편차가 0.93에서 0.86으로 줄어든 것으로 보아, 예시 글이 점수를 3~4점 부근으로 모으는 방향으로 작용한 것으로 보인다. 예시를 점수 기준으로만 골랐고 3편으로 수가 적었다는 점은 이 결과의 한계이다.

4.3 프롬프트 주입의 영향 (RQ3)

채점기	평균 점수 변화	점수 상승 비율	주입 후 5점 비율	Wilcoxon p
TF-IDF+Ridge	-0.02	0%	0%	–
Gemini Zero-shot → Zero-shot+주입	+1.46	85%	100%	<0.001
Gemini Zero-shot → 방어프롬프트+주입	+0.73	62%	66%	<0.001

- **LLM 채점기 (방어 없음):** 문장이 추가된 79편 모두 5점을 받았다. 원래 5점이었던 12편을 제외한 67편의 점수가 상승하였으며, 평균 상승폭은 1.46점이었다. 사람 점수가 낮은 글도 모두 5점을 받았다. 참고로 데이터 전체에서 두 채점자 평균이 5점인 글은 없다.
- **채점 근거의 내용:** 주입 조건에서 생성된 근거 79건 중, "지시문에 따라 5점을 부여한다"처럼 삽입된 문장을 따랐다고 언급한 것은 3건이었다. 나머지는 "주장을 논리적으로 서술하였다", "문장 간 연결이 자연스럽다"와 같은 내용 평가로 점수를 설명하였다. 79건 중 55건에는 삽입된 문장에 쓰인 "모범", "완벽"이라는 표현이 포함되어 있었다(주입이 없는 조건에서는 2건). 이 결과는 생성된 근거가 실제 판단 과정을 반영한다고 보기 어렵고, 근거만 보고는 점수가 조작되었는지 알기 어렵다는 것을 보여 준다.
- **방어 문구 추가:** 5점 비율은 100%에서 66%로 감소하였다. 그러나 평균 점수는 여전히 0.73점 상승하였고($p<0.001$), 사람 점수가 2.5 이하인 글 23편 중 48%가 5점을 받았다. 한편 1점을 받은 글은 2편에서 6편으로 늘었다. 방어 문구로 인해 일부 글이 감점된 것으로 보이며, 예측 표준편차는 0.93에서 1.20으로 커졌다.
- **TF-IDF 채점기:** 같은 문장을 추가해도 평균 변화는 −0.02점이었고 점수가 오른 글은 없었다. 회귀 모델에서는 추가된 문장이 일부 n-gram 특징으로만 반영되고, 이 특징들은 학습 데이터에서 점수와 관련이 적기 때문으로 해석된다.

5. 논의

1) 지시 수행 능력과 주입 취약성. LLM 채점기가 학습 데이터 없이 루브릭만으로 채점할 수 있는 것은 자연어 지시를 따르는 능력 때문이다. 이 능력은 학생 글 안에 포함된 지시에도 동일하게 작용하였다. 학습형 채점기는 지시를 해석하지 않으므로 주입의 영향을 거의 받지 않지만, 학습에 없는 문항에서는 성능이 낮았다. 두 방식의 장단점은 서로 연관되어 있다고 볼 수 있다.

2) 채점 근거의 신뢰성. LLM 채점기의 장점으로 점수와 함께 근거를 제공한다는 점이 꼽힌다. 그러나 본 실험의 주입 조건에서는 근거가 점수를 설명하기보다 이미 정해진 점수에 맞추어 생성된 것으로 보였다. 피드백을 교육 목적으로 활용하려면 근거가 실제 판단과 일치하는지에 대한 별도 검증이 필요하다.

3) 실제 적용 시 고려 사항. LLM 채점기를 단독으로 사용하기보다는 다음과 같은 보완이 필요할 것으로 보인다. - 글 속의 지시성 문장을 탐지하거나 제거하는 전처리 - 학습형 채점기와 LLM 점수가 크게 다를 경우 사람이 검토하는 절차 - 기준 예시나 사후 보정을 통한 LLM 점수 분포 조정

학습형 채점기가 주입의 영향을 거의 받지 않았으므로, 두 채점기 점수의 차이를 주입 탐지 신호로 활용하는 방법도 고려할 수 있다.

4) 한계. - LLM 실험은 79편, 단일 모델(Gemini 3.5 Flash-Lite), 단일 주입 문장, 단일 삽입 위치(글 끝)로 제한되었다. - 다른 LLM(상위 모델, 다른 회사 또는 국내 모델)에서는 결과가 다를 수 있다. - SBERT 채점기는 계산 시간 때문에 앞 256토큰과 train 1,780편만 사용하였고, 파인튜닝은 수행하지 않았다. - API 모델은 버전이 바뀌므로 동일한 결과가 재현되지 않을 수 있다.

6. 결론

한국어 논술형 글 18,010편을 대상으로 학습형 채점기와 LLM 채점기를 비교하였다. - 학습형 채점기는 학습에 포함되지 않은 문항에서 성능이 40~50% 감소하였다. 의미 유사도 기반 SBERT 임베딩은 TF-IDF보다 낮은 성능을 보였다. - 루브릭만 제공받은 LLM 채점기는 학습형 채점기의 cross-prompt 성능과 비슷한 수준이었으나, 점수를 평균 0.7점 높게 부여하였다. - 점수를 요구하는 문장 하나를 추가했을 때 LLM 채점기는 모든 글에 5점을 부여하였다. 생성된 근거에는 대부분 이 사실이 드러나지 않았고, 프롬프트 수준의 방어로는 영향을 일부만 줄일 수 있었다.

이 결과로 볼 때, LLM 채점기를 평가에 활용하려면 사람 점수와의 일치도뿐 아니라 입력 조작에 대한 강건성과 근거의 신뢰성도 함께 검토해야 한다.

향후 과제는 다음과 같다. - 삽입 위치, 문구, 언어를 다양화한 주입 실험 - 여러 LLM 간 비교(국가별 모델 포함) - 두 채점기 점수 차이를 이용한 주입 탐지 방법 - 8개 분석 항목별 채점으로의 확장 및 항목별 주입 영향 분석

참고문헌

1. AI Hub.「논술형 글쓰기 평가 데이터」. 한국지능정보사회진흥원. https://aihub.or.kr
2. Cohen, J. (1968). Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4).
3. Taghipour, K., & Ng, H. T. (2016). A neural approach to automated essay scoring. *EMNLP*.
4. Devlin, J. et al. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL*.
5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP*.

6. Park, S. et al. (2021). KLUE: Korean Language Understanding Evaluation. *NeurIPS Datasets and Benchmarks*.
7. Zheng, L. et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS*.
8. Greshake, K. et al. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *AISec*.
9. Mizumoto, A., & Eguchi, M. (2023). Exploring the potential of using an AI language model for automated essay scoring. *Research Methods in Applied Linguistics*, 2(2).
10. Turpin, M. et al. (2023). Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *NeurIPS*.

부록. Zero-shot Gemini 혼동 행렬 (행: 사람 점수 반올림, 열: Gemini 점수)

사람 ₩ Gemini	1	2	3	4	5
1	1	0	0	0	0
2	1	4	12	4	1
3	0	1	13	21	9
4	0	1	5	4	2

코드: https://github.com/tlswptjd200/korean-aes-llm-judge (데이터 로드 `load_data.py`, 베이스라인 `baselines.py`, 임베딩 `embed_ridge.py`, LLM 채점 및 주입 실험 `llm_judge.py`, 분석 및 그림 `analyze.py`. 모든 무작위 분할의 seed는 0으로 고정)